Telemedicine Platform — Interview Preparation Guide

What I built, why I built it, how it works, why these technologies, and what happens when something fails

Project: Telemed — video consultations with AI-assisted documentation
Stack: React + Vite · FastAPI · MySQL · JWT · WebSocket · WebRTC · STUN/TURN · Groq Whisper · Google Gemini
Level: fresher / early-career developer interviews

Answers are written to be spoken: in our application we use…, the reason we chose this was…, one limitation is…, if this fails, our fallback is…

Contents

Project Overview . . . 3

The Business Problem . . . 4

Complete Application Flow . . . 5

- Doctor verification . . . 5
- Appointment flow . . . 5
- Consultation lifecycle . . . 6

AI Documentation: Problem and Solution . . . 7

- What the structured draft contains . . . 7
- Design decisions worth explaining . . . 7

Architecture Overview . . . 8

Technology Stack and Why Each Was Chosen . . . 9

Level 1 — Business and Project Understanding . . . 10

Level 2 — Technology Questions . . . 12

- Python — 30 Questions . . . 12
- React — 30 Questions . . . 15
- FastAPI — 30 Questions . . . 18
- JWT — 30 Questions . . . 21
- STUN — 30 Questions . . . 24
- TURN — 30 Questions . . . 27

Project-Specific Architecture Questions . . . 31

AI, Whisper and Gemini Questions . . . 33

WebRTC Scenario Questions . . . 36

Security Questions . . . 38

Failure and Troubleshooting Questions . . . 39

Scaling and Future Improvement Questions . . . 40

Final Rapid-Fire Revision . . . 41

Project Overview

Telemed is a telemedicine video consultation platform. Patients find verified doctors, book a real date and time slot, and meet the doctor in a browser-based video call. After the call, the doctor documents the consultation, optionally with AI assistance, and the approved record appears in the patient's history. An administrator verifies doctors and manages the platform.

There are three kinds of users, and each has its own part of the application:

Role	What they do in the application
Patient	Registers, maintains a medical profile, searches doctors by name/specialization/language/experience, books a slot with a reason for visit, joins the video consultation, reads approved visit summaries.
Doctor	Registers with license and qualification details, waits for verification, sets weekly hours and days off, accepts or declines requests, sees relevant patient information, runs the call, reviews and approves the AI-assisted notes.
Admin	Verifies, rejects or suspends doctors, activates/deactivates accounts, manages appointments, consultation records and the list of specializations.

How to describe it in one breath:

> *I built a telemedicine platform where patients book verified doctors and consult them over a WebRTC video call. The interesting parts are the scheduling logic that prevents double booking, the WebSocket signaling for the video call, and an AI-assisted documentation feature that turns the consultation audio into a structured draft the doctor reviews and approves, so the doctor spends less consultation time typing.*

The Business Problem

Connecting a patient to a doctor online is only half of the problem. The other half is that **consultation time is limited**. In a 15–30 minute consultation the doctor has to:

- understand the problem and ask questions
- review the patient's history
- discuss symptoms and make a clinical decision
- explain the treatment
- document the consultation and write notes or a prescription

The last item, documentation, eats into the time the patient is paying for. Every minute spent typing is a minute the doctor is not listening to or looking at the patient. That is the bottleneck this project targets with AI-assisted documentation.

> The key sentence for interviews: *We use AI to reduce the time the doctor spends manually documenting the conversation, so more of the consultation is focused on the patient. The AI drafts; the doctor decides.*

A second, quieter problem is **trust**. A patient booking online cannot check a doctor's credentials themselves, so the platform verifies every doctor's license and qualifications before the doctor becomes visible or bookable.

Complete Application Flow

Doctor verification

```
Doctor registers (name, email, password)
        |
        v
Professional details (specialization, license / registration ID,
qualifications, years of practice)
        |
        v
Status = PENDING  -->  hidden from search, cannot be booked
        |
        v
Admin reviews credentials
        |
   +----+---------------------+
   v                          v
VERIFIED                  REJECTED (note sent to doctor)
visible + bookable        doctor can fix profile and resubmit
   |
   v
SUSPENDED (admin action, with note) -> hidden again
```

If a verified doctor later changes their license number or specialization, the profile automatically goes back to PENDING and admins are notified, because the thing that was verified has changed.

Appointment flow

```
Patient: Find Doctor -> Doctor Profile -> pick date -> pick time slot
        -> reason for visit -> Book
                     |
                     v
              Appointment PENDING (slot is already held)
                     |
        Doctor:  Accept --------------- Decline
                     |                     |
                     v                     v
               CONFIRMED              REJECTED (slot freed)
                     |
       15 min before start: "Join consultation" appears
                     |
                     v
          Video consultation (WAITING -> ACTIVE -> ENDED)
                     |
                     v
              Appointment COMPLETED
                     |
                     v
     Documentation: transcript -> AI draft -> doctor review -> APPROVED
                     |
                     v
          Visible in the patient's consultation history

  Either side may CANCEL a pending/confirmed appointment before it starts.
```

Consultation lifecycle

State	Meaning
(no record)	Appointment is confirmed but nobody has opened the room yet.
WAITING	First participant opened the room; the other one gets a notification.
ACTIVE	Both participants have joined at least once; started_at is recorded.
ENDED	Either side pressed End; ended_at is recorded and the appointment becomes COMPLETED.

AI Documentation: Problem and Solution

```
Doctor + Patient video consultation
        |   doctor presses "Record" (patient sees a recording indicator)
        v
Conversation audio, recorded in the doctor's browser as TWO tracks:
   doctor microphone  +  patient's incoming audio
        |   uploaded after the call ends
        v
Groq Whisper API (whisper-large-v3)  -> transcript per track
        |   merged by timestamp into Doctor / Patient turns
        v
Transcript (kept as spoken: English, Hindi or Hinglish)
        |
        v
Google Gemini API (gemini-2.5-flash) -> structured JSON draft
        |
        v
Doctor reviews / edits every section, adds own notes
        |
        v
Doctor approves  ->  Final consultation record  ->  patient history
```

What the structured draft contains

Chief complaint, symptoms, relevant medical history, current medications, allergies, important observations, assessment/diagnosis discussed, treatment/medication plan, and follow-up instructions. A section that was not discussed stays empty.

Design decisions worth explaining

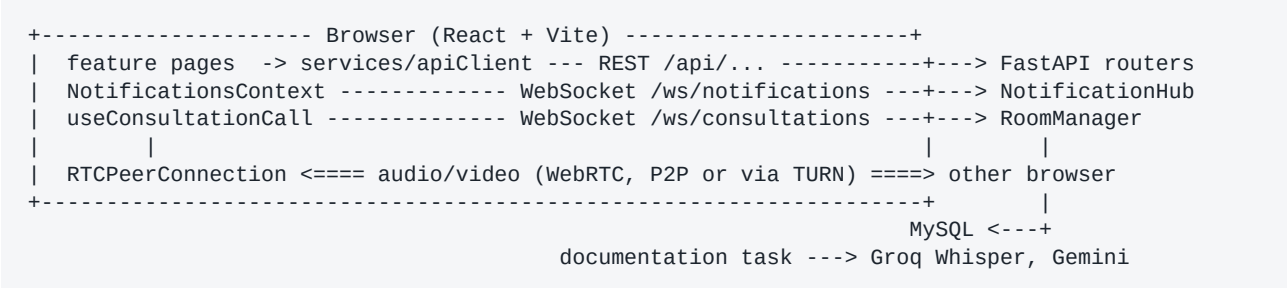
- **Two recordings instead of speaker detection.** One file is the doctor's microphone, the other is the patient's audio arriving over WebRTC. Every transcript line is therefore labelled Doctor or Patient correctly without a separate diarization model.
- **Language is not forced.** Whisper auto-detects the language, and we never use its translation mode, so Hinglish stays Hinglish in the transcript. Gemini then writes the clinical summary in English.
- **Three copies of the summary.** The raw AI output, the doctor's working draft and the approved version are stored separately. Only the approved version reaches the patient.
- **Runs in the background.** The upload returns immediately; transcription and summarization run as a background task, so the rest of the API is never blocked.
- **Fails safely.** If Groq or Gemini fails, the status records which stage failed, the doctor can retry, and manual notes always work. The consultation itself never depends on AI.
- **Consent and privacy.** Recording is off by default, the patient sees an indicator while it is on, the audio is stored outside the public folder and deleted as soon as it is transcribed.

Example of a Hinglish line the system must handle:

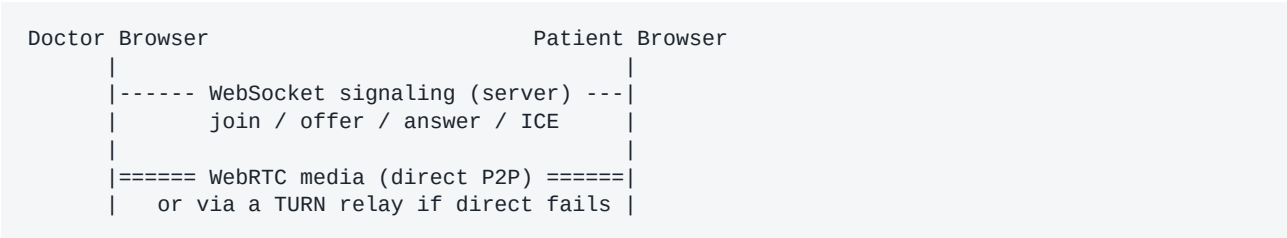
> *"Mujhe last three days se chest mein thoda pain ho raha hai, especially jab main walk karta hoon."*

The transcript keeps this sentence as spoken. The approved summary would read something like *Chief complaint: chest pain for three days, worse on walking* – and only if the patient actually said it.

Architecture Overview



- **REST** for normal data: auth, doctors, appointments, history, admin.
- **WebSocket** for two real-time needs: pushing notifications and WebRTC signaling.
- **WebRTC** for the audio and video itself, which flows between the two browsers and never through the server.
- **MySQL** as the single source of truth; every timestamp is stored in UTC.
- **Feature-first structure** on both sides: each feature (auth, doctors, patients, appointments, consultations, documentation, notifications, admin) owns its routes, models, schemas and service logic in the backend, and its pages, components, hooks and api.js in the frontend.



Technology Stack and Why Each Was Chosen

Layer	Technology	Why it fits this project
Frontend	React + JavaScript + Vite	Component model suits a multi-role dashboard; hooks keep call and media logic reusable; Vite gives a fast dev server and a proxy so the browser talks to one origin.
Backend	Python + FastAPI	Request validation from type hints (Pydantic), dependency injection for auth and roles, built-in WebSockets and automatic API docs at /docs.
Database	MySQL	Relational data with clear relationships (users, appointments, consultations); transactions and row locks are needed to prevent double booking.
Authentication	JWT	Stateless tokens that work the same for REST calls and WebSocket handshakes; the role travels with the token for the UI.
Signaling	FastAPI WebSocket	WebRTC needs a channel to exchange SDP and ICE candidates in real time; WebSocket is bidirectional and already part of FastAPI.
Audio/Video	WebRTC	Built into every modern browser, encrypted, low latency, and peer-to-peer so media does not load our server.
Connectivity	STUN + TURN	STUN lets peers discover public addresses for a direct path; TURN relays media when a direct path is impossible.
Speech-to-text	Groq Whisper API	Whisper handles Hindi, English and code-mixed speech well; Groq hosts it with very fast inference, so no GPU is needed locally.
AI documentation	Google Gemini API	Follows instructions well, supports a JSON response schema, and handles Hinglish input while writing English clinical notes.

Level 1 — Business and Project Understanding

These are the opening questions. The interviewer wants to know whether you understand what you built and why, before going into technology. Answer in terms of users and problems first.

Q1. What problem does your project solve?

It lets patients consult verified doctors over video without travelling, from finding a doctor to booking a real time slot to the call itself. The specific problem I focused on is that consultation time is limited and a part of it goes into the doctor typing notes. So besides the booking and video call, the project has AI-assisted documentation that drafts the notes from the conversation, which the doctor reviews and approves.

Q2. Who are the users of the application?

There are three roles: patients, doctors and administrators. Patients book and attend consultations, doctors manage their availability, consultations and documentation, and admins verify doctors and manage the platform. Each role has its own navigation and the backend enforces what each role is allowed to do.

Q3. Walk me through the complete patient journey.

The patient registers, fills in their medical information like allergies and current medicines, then goes to Find Doctors. They can search by name or specialization and filter by language, experience and availability. On a doctor's profile they pick a date and a free slot, enter the reason for the visit and send the request. When the doctor accepts, they get a real-time notification. Fifteen minutes before the start a Join button appears, they join the video call, and after the doctor approves the notes they can read the visit summary in their history.

Q4. And the doctor journey?

The doctor registers with their specialization, license number and qualifications, and stays pending until an admin verifies them. Meanwhile they complete their profile and set weekly hours and days off. Once verified, patients can book them; they accept or decline requests and can see the relevant patient's medical information. They run the video call, optionally record it, and afterwards review the AI draft, edit it and approve it as the final record.

Q5. Why is doctor verification required?

From a business view, patients are trusting the platform's list of doctors, so an unverified person giving medical advice would be a serious safety and legal risk. From a security view, anyone can create an account, so registration alone must never grant the right to see patients' medical data. That is why a new doctor is pending, hidden from search and cannot be booked until an admin checks the license and approves them.

Q6. Why did you need appointment scheduling instead of just calling an available doctor?

Doctors work in fixed hours and a consultation needs both people at the same time. Scheduling gives the doctor control over their time and gives the patient a guaranteed slot. It also gives us a record to attach the consultation and its documentation to, and it lets us control who can join which call and when.

Q7. Why did you introduce AI into the project?

Because documentation takes a noticeable part of the consultation time and the patient is paying for that time. If the conversation can be transcribed and turned into a structured draft, the doctor only has to check and correct it instead of writing it from scratch. That leaves more of the call for talking to the patient.

Q8. So what exactly is the business value of the AI feature?

More of the paid consultation time goes to the patient, notes are more complete because nothing has to be remembered and typed later, and the format is consistent across doctors. It does not replace the doctor's judgement: the doctor still reviews and approves everything.

Q9. Does the AI diagnose the patient or prescribe medicine?

No. It only summarizes what was said in the conversation. The prompt tells it to use only the transcript and leave a section empty if it was not discussed, and the assessment and plan sections only contain what the doctor actually said. Nothing reaches the patient until the doctor approves it.

Q10. What happens after a consultation ends?

The appointment is marked completed and the consultation gets its end time and duration. If the doctor recorded the call, the audio is uploaded and processed in the background, and the doctor sees the progress: upload, transcription, draft, review. The doctor edits and approves, and the patient gets a notification that the visit summary is available.

Q11. What happens if the AI documentation fails?

The consultation is already complete, so nothing about the visit is lost. The documentation status becomes failed with the stage that failed, the doctor sees the error and a Retry button, and they can always write and approve the notes manually. If the transcript succeeded but the summary failed, the transcript is kept and a retry only repeats the summary step.

Q12. What happens if the doctor doesn't record the call?

Then there is simply no AI draft. The doctor writes notes manually, like in a normal system. Recording is optional and off by default.

Q13. How does the patient know the call is being recorded?

When the doctor starts recording, a message is sent through the signaling channel and the patient's screen shows a 'Recorded for clinical notes' indicator for as long as recording is on. It is re-sent if the patient reconnects, so they always see the current state.

Q14. Why does multilingual and Hinglish support matter?

In India many consultations are not purely English or purely Hindi; people switch mid-sentence, like 'chest mein thoda pain ho raha hai'. If the system forced everything into English during transcription, it could lose meaning or misrecord symptoms. So we keep the transcript as spoken and let Gemini write the clinical summary in English.

Q15. What can the admin do and why is it needed?

The admin verifies or rejects doctors, suspends them if needed, deactivates accounts, cancels appointments, sees consultation records and manages the specialization list. Someone has to own trust and data quality on the platform. The admin intentionally does not see medical details, notes or transcripts, because managing the platform doesn't require them.

Q16. What would you say are the limitations of your solution?

There is no payment step yet, and recording happens in the doctor's browser, so if the doctor closes the tab before ending the call that recording is lost; manual notes still work. The real-time parts keep state in memory, so running several backend servers would need a shared store. And the AI quality depends on audio quality, which is why the doctor's review is mandatory.

Q17. How did you make sure two patients cannot book the same slot?

Every booking locks the doctor's row in the database, rechecks the slot and only then inserts the appointment. A second request at the same moment waits for the lock, sees the new appointment and gets a 'slot no longer available' error. I tested it by sending eight booking requests at once for the same slot, and exactly one succeeded.

Level 2 — Technology Questions

Six topics, thirty questions each, in the order an interviewer usually goes deeper. Questions are grouped into short follow-up chains: if you answer the first one well, expect the next.

Answer pattern that works for almost every question: **what it is**, **why it's used**, **how it works in this project**, and **why it was chosen** when relevant.

Python — 30 Questions

Python questions usually start with fundamentals and then check whether you can connect them to real code. Where possible, point to where the concept appears in your backend.

FUNDAMENTALS

Q1. Why did you choose Python for the backend?

Python is readable and quick to develop in, and it has strong libraries for exactly what I needed: FastAPI for the API, SQLAlchemy for MySQL, bcrypt and PyJWT for authentication, and httpx to call the AI services. It also made the AI integration natural, since most AI tooling has first-class Python support.

Q2. What is the difference between a list, a tuple, a set and a dict? Where did you use them?

A list is ordered and mutable, a tuple is ordered and immutable, a set holds unique values, and a dict maps keys to values. For example, the allowed appointment states that block a slot are a tuple because they never change; the language filter uses a set to remove duplicates; and the signaling server keeps rooms as a dict from room id to the participants in that room.

Q3. What are mutable and immutable objects, and why does it matter?

Mutable objects like lists and dicts can be changed in place; immutable ones like strings, tuples and ints cannot. It matters because a mutable object passed to a function or used as a default argument can be changed unexpectedly. For example, a default like def f(items=[]) is shared between calls, so I use None as the default instead.

Q4. What are type hints and do they matter at runtime?

Type hints describe the expected types, like start: datetime or -> list[Slot]. Plain Python ignores them at runtime, but FastAPI and Pydantic actually read them: the types on a route function decide how the request is parsed and validated. So in this project they are not just documentation, they drive validation.

FUNCTIONS AND OOP

Q5. What are *args and **kwargs?

*args collects extra positional arguments into a tuple and **kwargs collects extra keyword arguments into a dict. They are useful for wrappers and generic helpers. In the project, require_roles(*roles) accepts any number of roles, so a route can allow patients and doctors together.

Q6. What is a closure? Where is one used in your code?

A closure is an inner function that remembers variables from the function that created it. require_roles is an example: it returns a checker function that remembers which roles are allowed, and FastAPI calls that checker for each request.

Q7. What is a decorator?

A decorator wraps a function or class to add behaviour without changing its code. FastAPI routes are decorators, like @router.post('/appointments'), which register the function as an endpoint. I also use @property, @classmethod, Pydantic's @field_validator and @lru_cache to create the settings object only once.

Q8. Explain classes and objects using your project.

A class is a blueprint and an object is an instance of it. My database tables are classes, like User or Appointment, and each row is an object. There are also plain classes for in-memory state, like the RoomManager that tracks who is in each consultation room.

Q9. What are the four pillars of OOP and do you use them?

Encapsulation, abstraction, inheritance and polymorphism. Encapsulation: RoomManager hides its internal dict behind join, leave and peer_of methods. Inheritance: all models inherit from a common Base, and many share a TimestampMixin. Abstraction: route handlers call a service function like book() without knowing the locking details. Polymorphism is less central here, but Pydantic schemas extending other schemas use it.

Q10. What is a mixin? Why did you use TimestampMixin?

A mixin is a small class that adds a specific capability to other classes through inheritance. TimestampMixin adds created_at and updated_at columns, so every model that needs them inherits it instead of repeating the same two column definitions.

Q11. What is a @property? Give an example.

It lets a method be read like an attribute. Consultation.duration_seconds is a property that calculates the duration from start and end times, and Consultation.summary returns the approved AI summary only if the documentation status is approved. Pydantic can read these properties when building responses.

Q12. What are dataclasses and why did you use them?

A dataclass automatically generates __init__, __repr__ and comparison methods for a class that mostly holds data. I used them for small internal structures like Slot (start, end, available) and Participant in the signaling room. Pydantic models are used where data crosses the API boundary and needs validation.

Q13. What is an Enum and why are statuses enums instead of plain strings?

An Enum is a fixed set of named values. Appointment, consultation and verification statuses are enums so a typo like 'confimed' is impossible, the valid values are documented in one place, and MySQL stores them as an ENUM column. They inherit from str so they serialize to JSON as plain strings.

ERRORS, MODULES AND RESOURCES

Q14. How does exception handling work in Python?

Code that might fail goes in try, specific exceptions are handled in except, else runs if nothing failed, and finally always runs for cleanup. I catch specific exceptions rather than everything, for example IntegrityError when two requests create the same record, and httpx errors when calling Groq or Gemini.

Q15. Did you create custom exceptions? Why?

Yes, AIServiceError. When Groq or Gemini fails, the client code raises it with a message that is safe to show the doctor. The background pipeline catches only that type, marks the documentation as failed and records the stage. Unexpected bugs are not swallowed by accident.

Q16. How do HTTP errors work in your FastAPI code?

I raise HTTPException with the right status code: 404 when something doesn't exist or the user may not see it, 409 for conflicts like a slot that was just taken, 400 for bad input, 403 for the wrong role. FastAPI turns them into JSON responses, and the frontend shows the message to the user.

Q17. What is a context manager? Where do you use one?

It's an object used with 'with' that guarantees setup and cleanup. I use 'with SessionLocal() as db' in the WebSocket and background code so the database session is always closed, and 'with path.open("rb")' when sending audio files to Groq so the file is always closed.

Q18. What is a generator and where does yield appear in your backend?

A generator produces values lazily with yield. The get_db dependency is a generator: it creates a session, yields it to the route, and the code after yield closes it when the request finishes, even if the route raised an error.

Q19. What is the difference between a module and a package? How is your backend organized?

A module is one .py file and a package is a folder of modules with __init__.py. The backend is a package called app with sub-packages per feature, like app.features.appointments, each containing models.py, schemas.py, service.py and router.py. That keeps everything about one feature together.

Q20. What is a circular import and how did you avoid it?

It happens when two modules import each other and one isn't fully loaded yet. I avoid it by keeping a clear direction: routers import services, services import models, and models refer to other models by name as strings in SQLAlchemy relationships. One database module imports all models so SQLAlchemy can connect them.

Q21. How do you manage configuration and secrets in Python?

With pydantic-settings. A Settings class reads environment variables and the backend .env file, validates them (for example, the JWT secret must be at least 32 characters) and exposes typed values. No secrets are written in the code, and .env is git-ignored.

Q22. How do you handle dates and time zones?

Everything is stored in UTC, and inside the code every datetime is timezone-aware. Doctors enter their weekly hours in the clinic's time zone, so slot generation converts those local times to UTC with zoneinfo. I wrote a custom column type that always gives back aware UTC datetimes, so the API returns times with a 'Z' suffix.

ASYNC AND CONCURRENCY

Q23. What is async/await?

async def defines a coroutine, and await pauses it while waiting for I/O so the event loop can run other work. It lets one process serve many concurrent connections efficiently, which is ideal for WebSockets that mostly sit waiting for messages.

Q24. Are your FastAPI routes async? Why or why not?

Most REST routes are normal def functions because the database driver I use is synchronous. FastAPI runs def routes in a thread pool, so a blocking query doesn't freeze the server. The WebSocket handlers are async because they wait on sockets, and when they need the database they call run_in_threadpool.

Q25. What would go wrong if you called a blocking database query directly inside an async function?

It would block the event loop, so every other WebSocket and async request would freeze until that query finished. That's why the signaling code moves database work into a thread with run_in_threadpool.

Q26. How do you send a WebSocket notification from synchronous code?

Route handlers run in worker threads, but sockets belong to the event loop. The notification hub remembers the loop at startup and uses asyncio.run_coroutine_threadsafe to schedule the send on it. The route doesn't wait for it; it just hands the message over safely.

Q27. Where did you use asyncio.wait_for?

In WebSocket authentication. After accepting a socket, the server waits up to ten seconds for the first 'auth' message containing the JWT. If it doesn't arrive, the socket is closed, so unauthenticated connections can't stay open.

Q28. What is the GIL and does it affect your project?

The Global Interpreter Lock means only one thread runs Python bytecode at a time. It mainly limits CPU-heavy work. My backend is I/O-bound: it waits on MySQL, sockets and AI APIs, and threads release the GIL while waiting, so it isn't a practical bottleneck here.

Q29. How do you call external APIs like Groq and Gemini from Python?

With httpx. For Groq I send a multipart request with the audio file and the model name; for Gemini a JSON request with the transcript and a response schema. I set timeouts, check the status code, and convert any failure into an AIServiceError with a readable message.

Q30. How did you test the Python code?

With pytest. Pure functions like slot generation and transcript merging have unit tests. Integration tests use FastAPI's TestClient against a real MySQL test database and cover booking conflicts, concurrent booking, role checks, the full WebSocket signaling exchange and the AI pipeline, with the external AI calls replaced by fakes.

React — 30 Questions

COMPONENTS, PROPS AND STATE

Q1. Why did you use React for the frontend?

The application has many screens built from the same pieces: tables, status badges, forms, doctor cards. React's component model lets me build those once and reuse them. Hooks also let me package complex behaviour, like the video call, into reusable functions. With Vite, the development server starts instantly and proxies API calls to the backend.

Q2. What is a component?

A component is a function that returns UI for some input. In the project, Button, Card and Modal are generic UI components, DoctorListItem and SlotPicker belong to the doctors feature, and pages like FindDoctorsPage are also just components composed from smaller ones.

Q3. What is the difference between props and state?

Props are inputs passed from a parent and are read-only for the child. State is data a component owns and can change, which triggers a re-render. For example, BookingPanel keeps the selected slot and the reason text in state, and passes the doctor id and selected slot as props to SlotPicker.

Q4. What is JSX?

JSX is a syntax that looks like HTML inside JavaScript, which Vite compiles into React function calls. It lets me write the structure of a component next to its logic, for example rendering a Join button only when appointment.can_join is true.

Q5. How does conditional rendering work in your pages?

With normal JavaScript conditions in JSX. Most data pages follow one pattern: if loading, show a loading state; if error, show an error with a retry button; if the list is empty, show an empty state with a helpful action; otherwise show the data. That keeps every page consistent.

Q6. Why does React need a key when rendering lists?

The key tells React which item is which between renders, so it can update, move or remove the right element. I use database ids, like key={appointment.id}. Using the array index would cause bugs when items are reordered or removed.

Q7. What are controlled components?

Form inputs whose value comes from React state and change through onChange. All my forms work this way, which makes validation easy: for example, the registration form checks that passwords match before sending, and server field errors are shown next to the right input.

Q8. What does 'lifting state up' mean?

When two components need the same data, the state moves to their common parent. The selected slot lives in BookingPanel, not in SlotPicker, because both the slot picker and the 'Request appointment' button need it.

HOOKS

Q9. What are hooks? Which ones do you use?

Hooks let function components use state and lifecycle features. I use useState for local state, useEffect for side effects like loading data or opening sockets, useRef for values that shouldn't trigger re-renders such as the WebSocket or the video element, useCallback and useMemo for stable functions and context values, and useContext to read auth and notifications.

Q10. Explain useEffect and its cleanup function.

useEffect runs after render for side effects. Its dependency array controls when it re-runs, and the function it returns is the cleanup, run before the next effect or when the component unmounts. This is critical in the consultation page: cleanup stops the camera tracks, closes the RTCPeerConnection and closes the signaling socket, so nothing keeps running after the user leaves.

Q11. What is useRef used for in your project?

Two things. First, to reach DOM elements, like the video element where I set srcObject to the media stream. Second, to keep mutable values across renders without re-rendering, like the current WebSocket or the latest media state that the call logic needs to read.

Q12. What are custom hooks? Give examples.

A custom hook is a function starting with 'use' that combines other hooks into reusable logic. useApi loads data with loading/error state and a reload function. useLocalMedia handles camera and microphone. useConsultationCall handles signaling and the peer connection. useConsultationRecorder handles recording for AI documentation. Pages stay small because the logic lives in these hooks.

Q13. What is a stale closure and how did you avoid it?

An effect or callback can capture old state values from the render where it was created. For example, the notification subscription would call an old version of a page's reload function. I keep the latest handler in a ref, updated on every render, so the long-lived subscription always calls the current function.

Q14. Why does React StrictMode run effects twice in development, and how did you handle it?

StrictMode mounts, unmounts and mounts again to expose effects that don't clean up properly. In a video call that means the camera and socket open, close and open again. Because every effect has a proper cleanup and checks a 'disposed' flag after each await, the second run works correctly and nothing leaks.

Q15. When would you use useCallback or useMemo?

When a function or object is passed to children or used as an effect dependency, and recreating it on every render would cause unnecessary re-renders or re-runs. For example, the auth context value is memoized so every component using it doesn't re-render on unrelated updates.

CONTEXT AND STATE MANAGEMENT

Q16. What is React Context and what do you keep in it?

Context shares data with many components without passing props through every level. I have three: AuthContext for the signed-in user and login/logout, NotificationsContext for live notifications, and ToastContext for short messages. Everything else is local to the feature that needs it.

Q17. Why didn't you use Redux or another state library?

The app doesn't have much truly global state: the user, notifications and toasts. Everything else belongs to one page. Adding a state library would add boilerplate without solving a real problem. If the app grew a lot of shared, cached server data, I would consider one.

Q18. How does the app restore a session after a page refresh?

The token is in sessionStorage. On startup AuthContext calls /auth/me with it; if that succeeds the user is signed in, and if it fails the token is removed and the user goes to the login page. While this check runs, protected routes show a loader instead of redirecting too early.

Q19. Why sessionStorage instead of localStorage for the token?

sessionStorage is per tab and is cleared when the tab closes, which suits shared computers in a clinic. It also let me test calls by signing in as a patient in one tab and a doctor in another in the same browser. The trade-off is the same as localStorage: injected scripts could read it, so an httpOnly cookie would be the next hardening step.

API CALLS, ROUTING AND REAL TIME

Q20. How are API calls organized?

There is one shared apiClient that adds the base URL and JWT, sends JSON and turns FastAPI errors into a readable ApiError with field-level messages. Each feature has its own api.js with small functions like bookAppointment or getDoctorSlots, so components never build URLs or headers themselves.

Q21. How do you handle loading and error states consistently?

The useApi hook returns data, loading, error and reload. useAsyncAction wraps actions like 'Approve' with a pending flag and an error. Pages use these with shared LoadingState, ErrorState and EmptyState components, so every screen behaves the same way.

Q22. How does routing work and how are routes protected?

I use React Router. Routes are grouped by role under a ProtectedRoute that checks the signed-in user's role. A signed-out user is redirected to login and comes back afterwards; a user with the wrong role is sent to their own dashboard. This is only for user experience: the backend enforces the same rules on every request.

Q23. Why are search filters stored in the URL?

On Find Doctors, filters like specialization and language are query parameters. That means the browser's back button works, a filtered search can be bookmarked, and the patient dashboard can link directly to 'Cardiologists' with a prepared URL.

Q24. How does the frontend receive real-time notifications?

NotificationsContext opens a WebSocket after login, authenticates with the token, and adds incoming notifications to the list, shows a toast and informs subscribed pages. If the socket drops it reconnects with increasing delays, and after each reconnect it reloads the list so nothing is missed.

Q25. How does a page refresh itself when an appointment changes?

Pages subscribe with useNotificationEvents. For example, when a doctor accepts a request, the patient's appointments page receives the notification and calls reload, so the status changes from pending to confirmed without a manual refresh.

Q26. How did you build the video call UI in React?

The page has three states: a lobby with a camera preview, the in-call view and the ended view. useLocalMedia owns the camera stream, useConsultationCall owns the signaling socket and peer connection and exposes a phase like 'waiting' or 'connected'. The page just renders based on that phase, and VideoTile attaches a stream to a video element.

Q27. Why does the remote video element stay mounted when the camera is off?

The remote audio plays through the same video element. If I removed the element when the other person turned off their camera, their audio would stop too. So I hide the video visually and show an avatar on top.

Q28. How did you handle the browser blocking autoplay?

Browsers can block media with sound from playing automatically. The pre-join lobby means the user clicks Join, which counts as user interaction. If play() is still rejected, the tile shows an 'Enable audio' button that calls play() on click.

Q29. How did you make the UI responsive?

With plain CSS: layout grids that collapse to one column on smaller screens, a sidebar that becomes a slide-in menu, and tables inside a scrollable wrapper. The video room moves the details panel over the video on narrow screens.

Q30. How did you structure the frontend code?

Feature-first. Each feature folder (auth, doctors, patients, appointments, consultations, documentation, notifications, admin) contains its pages, components, hooks and api.js. Shared pieces live in components/ui, components/layout, hooks, services and context. When I work on appointments, almost everything I need is in one folder.

FastAPI — 30 Questions

BASICS AND WHY FASTAPI

Q1. What is FastAPI?

FastAPI is a Python web framework for building APIs. You declare routes as functions with type hints, and it uses those hints to parse and validate requests, serialize responses and generate interactive documentation.

Q2. Why was FastAPI a reasonable choice for this backend?

I needed REST endpoints, strict input validation for medical and scheduling data, role-based access, WebSockets for signaling and notifications, and background work for AI processing. FastAPI has all of that built in: Pydantic validation, dependency injection, WebSocket support and BackgroundTasks. The automatic /docs page also made testing endpoints easy.

Q3. How is your API organized?

Each feature has an APIRouter with a prefix, for example /appointments or /doctors, and main.py includes them all under /api. The WebSocket routers are mounted at /ws. Inside a feature, router.py handles HTTP details and calls service.py, which holds the business logic.

Q4. Why separate routers from services?

The router deals with HTTP: parameters, status codes, response models. The service deals with rules: can this slot be booked, can this doctor accept this appointment. Keeping them apart makes the rules easier to test and lets the admin router reuse the same cancel logic as patients and doctors.

Q5. What are path parameters and query parameters? Examples?

Path parameters identify a resource, like /appointments/{appointment_id}. Query parameters refine a request, like /doctors?specialization_id=2&language=Hindi&sort=experience. FastAPI converts and validates both from the type hints, for example limit must be between 1 and 50.

REQUESTS, RESPONSES AND VALIDATION

Q6. What is Pydantic and how is it used here?

Pydantic defines data models with types and rules. Request bodies like AppointmentCreate are Pydantic models, so a booking without a reason, or a reason shorter than five characters, is rejected with a 422 before my code runs. Responses use models too, which controls exactly which fields leave the server.

Q7. What is response_model and why is it important for a medical app?

response_model declares the shape of the response, and FastAPI filters the output to that shape. That prevents accidental leaks: for example, the password hash is never part of UserOut, and admin responses don't include reasons for visit or consultation notes.

Q8. Give an example of custom validation.

AvailabilityRuleIn checks that the start time is before the end time. VerificationUpdate requires a note when an admin rejects or suspends a doctor. The medical profile rejects a date of birth in the future. These produce clear 422 errors the frontend shows next to the field.

Q9. Which HTTP status codes do you use and when?

200 for normal reads and updates, 201 when something is created, like a booking, 202 when processing continues in the background, like the AI upload, 204 when there's nothing to return, 400 for invalid requests, 401 when not authenticated, 403 for the wrong role, 404 when a resource doesn't exist or isn't yours, 409 for conflicts like a taken slot, and 422 for validation errors.

Q10. Why do you sometimes return 404 instead of 403?

If a doctor asks for a patient they have no appointment with, returning 403 would confirm that patient id exists. Returning 404 reveals nothing. The same applies to appointments that belong to someone else.

Q11. How do you handle file uploads?

With UploadFile. Profile photos are limited to JPEG, PNG or WebP and 2 MB, and saved with a random file name. Consultation recordings are uploaded as several files with matching 'speakers' and 'offsets' form fields; they are size-limited and stored in a private folder, not served publicly.

DEPENDENCIES AND AUTHENTICATION

Q12. What is dependency injection in FastAPI?

A route declares what it needs with Depends, and FastAPI provides it before the route runs. My routes depend on get_db for a database session and on get_current_user or a role check for the user. If the dependency raises an error, like 401, the route never runs.

Q13. How does get_db work?

It's a generator dependency: it opens a SQLAlchemy session, yields it to the route, and closes it afterwards in a finally block. Every request gets its own session, and it's always closed even if the route fails.

Q14. How do you protect an endpoint so only doctors can use it?

With a dependency. require_roles(UserRole.DOCTOR) first runs get_current_user to validate the token, then checks the role and raises 403 if it doesn't match. I wrapped these into type aliases like DoctorUser, so a route just declares doctor: DoctorUser. The admin router applies the admin check once for all its routes.

Q15. Why reload the user from the database on every request instead of trusting the token?

If an admin deactivates an account, existing tokens would otherwise keep working until they expire. Loading the user each time means deactivation takes effect immediately, and the role used for authorization always comes from the database.

SYNC, ASYNC AND BACKGROUND WORK

Q16. What is the difference between def and async def routes in FastAPI?

async def routes run on the event loop and must not block. def routes run in a thread pool, so blocking work like a synchronous database query is fine there. Since I use synchronous SQLAlchemy, most routes are def.

Q17. What are BackgroundTasks and where did you use them?

They run a function after the response is sent. When the doctor uploads recordings, the endpoint saves the files, returns 202 immediately, and a background task calls Groq and then Gemini. The doctor's page polls the status, so the API is never blocked waiting for AI.

Q18. What are the limits of BackgroundTasks?

They run in the same server process, so if the server restarts mid-task, the work is lost. I handle that by allowing a retry if processing has been stuck for over ten minutes. At larger scale I would move this to a proper job queue with automatic retries.

Q19. What is the lifespan function in main.py for?

It runs code at startup and shutdown. At startup I store the running event loop in the notification hub, so synchronous route handlers in worker threads can schedule WebSocket messages on that loop.

WEBSOCKETS

Q20. How do WebSockets work in FastAPI?

A WebSocket route is an async function that accepts the connection and then loops: receive a message, handle it, send messages back. It stays open for the whole session. I have two: /ws/notifications for pushing notifications and /ws/consultations/{room_id} for call signaling.

Q21. How are WebSocket connections authenticated?

Browsers can't attach an Authorization header to a WebSocket, and putting the token in the URL would leak it into server logs. So the server accepts the connection and waits for the first message to be {type: 'auth', token}. It validates the JWT and loads the user; if anything is wrong within ten seconds, it closes the socket with code 4401.

Q22. What does the signaling WebSocket actually do?

It checks that the user belongs to that consultation, adds them to the room, tells the other participant that someone joined, and relays offer, answer and ICE candidate messages between the two. It also relays small state messages like mute or recording status, and handles the end-call message.

Q23. What message format do your sockets use?

Every message is JSON with a type and a data object, like {type: 'offer', data: {sdp: ...}}. The server adds a 'from' field when it relays a message. A consistent envelope makes both sides simple to write and debug.

Q24. What happens if the same user opens the call in two tabs?

The room allows one connection per user. The new connection replaces the old one, and the old tab receives an error message and is closed with a specific code, so the user sees 'you joined from another window' instead of two competing connections.

Q25. How do you handle invalid or oversized WebSocket messages?

Invalid JSON is ignored instead of crashing the handler, unknown types get an error message back, and relayed payloads larger than 64 KB are dropped. The room membership is checked once at connect time against the database.

ERRORS, CONFIG AND TESTING

Q26. How does error handling look from the frontend's point of view?

FastAPI returns errors as JSON with a 'detail' field. For HTTPException it's a message string; for validation errors it's a list with the field location. The frontend's apiClient converts both into one ApiError with a message and per-field errors.

Q27. How is CORS handled?

In development the Vite dev server proxies /api and /ws to FastAPI, so the browser only sees one origin and CORS isn't needed. The backend still has CORSMiddleware configured from an environment variable for setups where the frontend is served separately.

Q28. How do you start the database schema?

An init script creates the database if allowed, creates tables from the SQLAlchemy models and seeds specializations, the admin and demo data. It's safe to run repeatedly. For production I'd switch to proper migrations so schema changes are versioned.

Q29. How did you test your FastAPI endpoints?

With FastAPI's TestClient and pytest against a real MySQL test database. Tests register users, verify a doctor as admin, book and accept appointments, open two WebSocket connections and run the whole signaling exchange, and run the AI pipeline with fake Groq and Gemini responses.

Q30. Is there anything you would change in your FastAPI setup for production?

Run it behind HTTPS, add rate limiting on login, move background AI processing to a job queue, use database migrations, and move in-memory WebSocket state to a shared store if I run more than one server.

JWT — 30 Questions

WHAT IT IS AND WHY

Q1. What is a JWT?

A JSON Web Token is a compact, signed token that carries a few facts about the user, called claims. The server creates it at login and signs it with a secret; on later requests the client sends it back and the server checks the signature to trust those claims without a session lookup.

Q2. What are the three parts of a JWT?

Header, payload and signature, separated by dots and Base64URL-encoded. The header says which algorithm is used, the payload holds the claims, and the signature proves the first two parts were created by the server and not changed.

Q3. What claims does your token contain?

sub, the user id; role, which is patient, doctor or admin; iat, when it was issued; and exp, when it expires. I deliberately keep it small: no email, no medical data, nothing sensitive.

Q4. Is a JWT encrypted?

No, a normal JWT is only signed, not encrypted. Anyone holding the token can decode and read the payload. That's exactly why I never put sensitive information in it. The signature only guarantees it wasn't tampered with.

Q5. Why did you choose JWT for this application?

The same token works for REST calls and for both WebSocket connections, the server doesn't need a session table, and the role can travel with the token so the frontend can show the right navigation immediately. It's also a widely understood standard, which keeps the authentication easy to explain and maintain.

Q6. What is the difference between JWT and session-based authentication?

With sessions the server stores session data and the browser holds only a session id, usually in a cookie. With JWT the information is in the signed token itself and the server only verifies it. Sessions are easier to revoke instantly; JWTs are easier to use across different kinds of connections. I partially get the revocation benefit back by reloading the user from the database on each request.

LOGIN AND PROTECTED REQUESTS

Q7. Walk me through the login flow.

The user submits email and password to /api/auth/login. The backend finds the user, checks the password against the bcrypt hash and checks the account is active. It then creates a JWT with the user id and role, signs it with HS256 and the server secret, and returns the token together with the user profile. The frontend stores the token and redirects to that role's dashboard.

Q8. How are passwords stored?

As bcrypt hashes, never in plain text. bcrypt adds a random salt and is deliberately slow, which makes guessing passwords from a leaked database expensive. At login I compare the entered password with the stored hash.

Q9. Why do you return the same error for a wrong email and a wrong password?

If the messages were different, an attacker could find out which email addresses are registered. So both cases return 'Invalid email or password'.

Q10. How does the frontend send the token?

The shared apiClient adds an 'Authorization: Bearer <token>' header to every request. Components never deal with the token directly.

Q11. How does the backend identify the user on a protected request?

The get_current_user dependency reads the bearer token, verifies the signature and expiry with the secret, takes the user id from the sub claim and loads that user from the database. If the token is invalid or the user is missing or deactivated, it returns 401. The route receives the user object and works with that.

Q12. What happens when the token expires?

Tokens are valid for 8 hours by default (configurable). After that the backend returns 401. The frontend's API client has a global handler for 401 that signs the user out and sends them to the login page.

Q13. Do you use refresh tokens?

Not currently; the user signs in again after the token expires. For a production system I would add short-lived access tokens with a refresh token, so sessions can be kept alive safely and revoked.

SIGNING AND VALIDATION

Q14. What algorithm do you use and what does that mean?

HS256, which is HMAC with SHA-256 using a single shared secret. The same secret signs and verifies tokens, which is fine because only my backend creates and checks them. If other services needed to verify tokens without being able to create them, an asymmetric algorithm like RS256 would be better.

Q15. Where is the JWT secret stored?

In the JWT_SECRET environment variable in backend/.env, which is not committed. The settings class refuses to start if it's shorter than 32 characters, and the setup script generates a random one.

Q16. What happens if someone changes the role in the token from patient to admin?

The signature no longer matches, so verification fails and the request gets 401. And even with a valid token, my backend takes the role from the database user, not from the token, when checking permissions.

Q17. What checks happen during validation?

The signature must match, the algorithm must be the one I expect, the exp claim must be in the future, and the sub claim must be present and point to an existing, active user.

Q18. Why did you specify the allowed algorithm explicitly when decoding?

To avoid the classic attack where a token claims a different algorithm, like 'none'. The decode call only accepts HS256, so any other algorithm is rejected.

AUTHORIZATION AND ROLES

Q19. What is the difference between authentication and authorization?

Authentication answers 'who are you?', which the JWT check handles. Authorization answers 'what are you allowed to do?', which the role checks and ownership checks handle.

Q20. How is role-based access implemented?

With a FastAPI dependency, require_roles. A route declares, for example, doctor: DoctorUser, and the dependency rejects anyone whose role isn't doctor with 403. The whole admin router has the admin check applied once, so no admin endpoint can be added without protection by mistake.

Q21. Is checking the role enough?

No. A patient role allows reading appointments, but only their own. So services also check ownership: an appointment is only returned if the user is its patient or doctor. Role checks decide the kind of action; ownership checks decide which records.

Q22. How do you prevent a patient from reading another patient's history?

Patients never pass a patient id. History and appointment queries filter by the id of the signed-in user taken from the token and database. Asking for someone else's appointment by id returns 404 because the ownership check fails.

Q23. How can a doctor see a patient's medical information, and when not?

Only when there's a care relationship: at least one pending, confirmed or completed appointment between that doctor and patient. A declined or cancelled request doesn't count. Without that relationship the doctor gets 404, as if the patient didn't exist.

Q24. What can the admin see?

Everything needed to manage the platform: users, doctors' credentials, appointment times and statuses, consultation durations. Not medical profiles, reasons for visit, notes or AI transcripts. Admin is a management role, not part of the patient's care team.

Q25. If the role is in the token, why does the frontend also check it?

Only for user experience: to show the right menu and redirect users away from pages they can't use. The frontend can be modified by anyone, so the real protection is always in the backend.

SECURITY CONSIDERATIONS

Q26. Where is the token stored in the browser and what's the risk?

In sessionStorage. It's cleared when the tab closes, but like localStorage it can be read by JavaScript, so a cross-site scripting bug could steal it. React escapes rendered text by default, which lowers that risk. The stronger option would be an httpOnly cookie with CSRF protection.

Q27. How is the token used for WebSockets?

After the socket opens, the client sends {type: 'auth', token} as the first message. The server validates it the same way as for HTTP. I avoid putting the token in the URL because URLs end up in logs.

Q28. How do you revoke access for a user immediately?

The admin deactivates the account. Because every request reloads the user and checks is_active, the next request with the old token fails with 401, even though the token itself hasn't expired.

Q29. What happens if the JWT secret leaks?

Anyone could create valid tokens for any user, so it's a critical incident. You'd change the secret right away, which invalidates every existing token, so all users must sign in again. That's why the secret is only in the server environment and never in code.

Q30. What would you improve in the authentication?

Rate limiting on the login endpoint, refresh tokens with short-lived access tokens, httpOnly cookie storage, and optionally two-factor authentication for doctors and admins, since they can access sensitive data.

STUN — 30 Questions

Keep STUN answers practical: what problem it solves, what it gives the browser, and where it stops helping. Interviewers usually chain these into TURN questions.

NAT AND ADDRESSES

Q1. What is NAT?

Network Address Translation lets many devices on a private network share one public IP address. The router rewrites outgoing packets so they appear to come from its public address and maps replies back to the right device. It's why most home and office devices don't have their own public address.

Q2. What's the difference between a private and a public IP address?

A private address, like 192.168.1.10, only works inside the local network. A public address is reachable on the internet. A doctor's laptop at home usually only knows its private address, which is useless to a patient's browser somewhere else.

Q3. Why is NAT a problem for WebRTC?

WebRTC wants the two browsers to send media directly to each other. But each browser only knows its private address, and the NAT normally only lets in replies to connections started from inside. So the browsers need a way to learn their public address and open a path through the NAT.

WHAT STUN IS

Q4. What is STUN?

STUN, Session Traversal Utilities for NAT, is a simple server that tells a client what its public IP address and port look like from the internet. The browser asks the STUN server 'how do you see me?' and gets back its public address, which it can share with the other peer.

Q5. Why does WebRTC need STUN?

Without it, the browsers could only share private addresses, so a direct connection would only work on the same local network. With STUN they also learn their public addresses, which makes direct peer-to-peer connections possible across most home and office networks.

Q6. Does the STUN server carry any audio or video?

No. It only answers short address requests at the start (and during reconnection). Once the peers connect, media flows directly between them. That's why STUN servers are cheap to run and public ones exist.

Q7. Which STUN server does your project use?

By default a public Google STUN server, configured through the STUN_SERVER environment variable. Several URLs can be given separated by commas. The frontend receives this list from the backend when joining a call, so it's never hard-coded in the browser.

Q8. Why does the backend send the ICE server list instead of hard-coding it in React?

Configuration belongs in one place and can change per environment without rebuilding the frontend. More importantly, TURN credentials come in the same list, and those should only be given to authenticated users who are actually joining a consultation.

ICE AND CANDIDATES

Q9. What is ICE?

Interactive Connectivity Establishment is the process WebRTC uses to find a working network path. Each browser collects possible addresses, called candidates, exchanges them with the other side, and tests pairs of candidates until one works.

Q10. What types of ICE candidates are there?

Host candidates are the device's own local addresses. Server-reflexive candidates are the public addresses learned from STUN. Relay candidates are addresses on a TURN server. The browser tries direct ones first and uses relay only if needed.

Q11. How do the candidates reach the other browser in your project?

Through our signaling WebSocket. When the browser finds a candidate, the onicecandidate event fires, and we send it as an 'ice-candidate' message. The server relays it to the other participant, who adds it to their peer connection.

Q12. What is trickle ICE?

Sending candidates one by one as they're found, instead of waiting for all of them before sending the offer. It makes connections faster. We do this: candidates are sent as soon as onicecandidate fires.

Q13. What happens if a candidate arrives before the offer or answer has been applied?

addIceCandidate would fail because the peer connection doesn't have the remote description yet. In our hook, early candidates are queued and added right after setRemoteDescription completes.

Q14. What are connectivity checks?

Once both sides have candidates, ICE sends small test packets over candidate pairs to see which paths actually work, then picks the best working pair. That's when the connection state goes to 'connected'.

Q15. How does your app show whether the call is direct?

After connecting, we read the peer connection's statistics and look at the selected candidate pair. If neither side is a relay candidate, the status bar shows 'Direct connection'; otherwise 'Relayed via TURN'. We also show the round-trip time.

WHERE STUN STOPS WORKING

Q16. What are the limitations of STUN?

STUN only discovers addresses; it can't force traffic through. With symmetric NATs, which assign a different public port for each destination, the address learned from STUN isn't the one the other peer can use. Strict firewalls that block UDP also defeat it. In those cases a direct connection is impossible and TURN is needed.

Q17. What is a symmetric NAT?

A NAT that creates a different public mapping for every destination. The mapping it showed the STUN server won't accept packets from the other peer, so direct connection fails. It's common in some corporate networks and mobile carriers.

Q18. What percentage of calls typically work with STUN alone?

Most calls between ordinary home networks work with STUN alone. A minority, mainly involving corporate firewalls, symmetric NAT or some mobile networks, need TURN. That's why TURN is a necessary fallback even though it isn't the main path.

Q19. What happens in your app if STUN isn't reachable?

The browsers only have host candidates. On the same machine or local network the call still connects. Across different networks it fails; our app then tries ICE restarts, and finally shows 'Unable to connect' with a hint that a TURN server may be needed, plus a Try again button.

Q20. How did you test calls locally?

Two browser tabs on the same computer, signed in as patient and doctor. They connect through host candidates immediately and the status shows a direct connection. I also automated it with two headless Chrome sessions using fake camera devices and checked that video frames arrived on both sides.

Q21. Do you need STUN when both users are on the same Wi-Fi?

Not really; host candidates are enough on the same network. STUN matters when the users are on different networks, which is the normal case for telemedicine.

Q22. Does STUN have security concerns?

STUN itself only reveals your public address, which any server you connect to already sees. The bigger point is that WebRTC reveals IP addresses to the other peer. In a consultation that's between the doctor and patient; if hiding IPs mattered, you could force relay mode through TURN.

CONNECTION LIFECYCLE

Q23. What connection states do you handle?

'connected' starts the call timer and stats; 'disconnected' shows 'Reconnecting…' and waits a few seconds because it often recovers by itself; 'failed' triggers an ICE restart. If restarts don't help, the user sees a clear failure message with a retry option.

Q24. What is an ICE restart?

Gathering new candidates and re-running connectivity checks on the existing call, without hanging up. It's done by creating a new offer with iceRestart. In our app only the side that made the original offer restarts, at most twice, so both sides don't restart at the same time.

Q25. What happens if the user's network changes during a call, like Wi-Fi to mobile data?

The old candidate pair stops working, so the connection becomes disconnected and then possibly failed. Our app waits four seconds, then does an ICE restart, which gathers candidates for the new network and reconnects. If the signaling socket also dropped, it reconnects and the call is renegotiated.

Q26. Does STUN work over TCP?

STUN normally uses UDP, which is what WebRTC media prefers. If UDP is blocked entirely, STUN alone can't help. That's a case for TURN over TCP or TLS.

Q27. How do you configure multiple STUN servers?

STUN_SERVER accepts a comma-separated list; the backend splits it and includes all of them in the ICE server list given to RTCPeerConnection. Having more than one is useful if a public server is slow or unavailable.

Q28. What does the signaling server have to do with STUN?

Nothing directly. The browser contacts the STUN server itself. The signaling server only relays the candidates that resulted from it. Keeping those two roles separate is a common interview point.

Q29. Explain the full connection setup in order.

Both users join the room over WebSocket. The one already waiting creates an SDP offer and sends it; the other applies it and sends an SDP answer. Meanwhile both browsers ask STUN for their public addresses and trickle candidates through the WebSocket. ICE tests the pairs, picks a working one, and media starts flowing directly between the browsers.

Q30. If STUN gives you a public address, why can't you always connect?

Because knowing an address isn't the same as being allowed to send to it. Symmetric NATs and firewalls drop packets that don't match an existing outgoing connection to that exact destination. Then the only option is a server both sides can reach, which is TURN.

TURN — 30 Questions

WHAT AND WHY

Q1. What is TURN?

TURN, Traversal Using Relays around NAT, is a server that relays media between peers when they can't connect directly. Each browser sends its media to the TURN server, which forwards it to the other browser.

Q2. Why is TURN needed if you already have STUN?

STUN only helps the peers discover their public addresses. If a NAT or firewall still blocks direct traffic, knowing the address doesn't help. TURN gives both peers a reachable meeting point, so the call can still happen.

Q3. What is the difference between STUN and TURN in one sentence?

STUN tells you your public address so you can try to connect directly; TURN carries your media for you when you can't.

Q4. When is TURN used?

Only when no direct candidate pair works. ICE prefers host and STUN-based candidates and falls back to relay candidates. Typical cases are corporate networks, symmetric NAT, some mobile carriers and networks that block UDP.

Q5. Why is TURN a fallback and not the preferred path?

Relaying adds an extra hop, so latency is a bit higher, and every byte of video goes through the TURN server, which costs bandwidth and money. A direct path is faster and free for the provider. So TURN is used only when it's the only path that works.

Q6. Why is it important for a telemedicine product specifically?

A consultation that can't connect is a failed appointment for a patient who may be unwell, and doctors often work from hospital or clinic networks with strict firewalls. TURN is what makes the service reliable for those users.

CONFIGURATION IN THE PROJECT

Q7. How is TURN configured in your project?

Through TURN_SERVER, TURN_USERNAME and TURN_PASSWORD in the backend's .env. TURN_SERVER can contain several URLs, for example a UDP one and a TLS one. The backend adds them with the credentials to the ICE server list returned when a participant joins a call.

Q8. Why are TURN credentials needed?

Because a TURN server relays traffic and costs bandwidth, it must not be open to anyone on the internet, or people could use it as a free relay. Credentials restrict it to your application.

Q9. How do you protect the TURN credentials?

They live only in the backend environment and are given out only by the join endpoint, which requires a valid JWT and checks that the user belongs to that appointment. A user still sees them in their browser, which is a limitation of static credentials.

Q10. How would you improve that?

Use time-limited TURN credentials: the backend creates a username containing an expiry time and a password signed with a secret shared with the TURN server. Each user gets short-lived credentials, so leaking them doesn't matter for long.

Q11. What is ICE_TRANSPORT_POLICY=relay used for?

It forces the browser to use only relay candidates. I use it to prove TURN actually works: with relay forced, the status bar should show 'Relayed via TURN'. If no TURN server is configured, the backend ignores that setting, because forcing relay without TURN would make every call fail.

Q12. What are turn: and turns: URLs?

turn: uses UDP or TCP to the TURN server; turns: uses TLS, usually on port 443. TLS on 443 looks like normal HTTPS traffic, so it gets through very strict firewalls that block everything else.

Q13. Does TURN change your signaling?

No. Relay candidates are exchanged through the same 'ice-candidate' messages as other candidates. The signaling server doesn't know or care which path is eventually chosen.

HOW RELAYING WORKS

Q14. How does a browser get a relay candidate?

It authenticates with the TURN server and asks for an allocation, which is a public address and port on the TURN server reserved for this browser. That address becomes a relay candidate and is sent to the other peer like any other candidate.

Q15. Can the TURN server see the audio and video?

No. WebRTC media is encrypted end to end between the browsers, and the TURN server only forwards encrypted packets. It knows who is talking to whom and how much data, but not the content. That's important for patient privacy.

Q16. What are the performance implications of TURN?

Slightly higher latency because of the extra hop, and the TURN server's bandwidth becomes a limit: a video call can use around one to two megabits per second in each direction, all through the server. Placing TURN servers close to users reduces the latency impact.

Q17. How would you know how many calls go through TURN?

Our app already detects the route from the peer connection stats. In production I'd send that, with the round-trip time, to the backend when the call connects, to see what share of consultations need relay and plan TURN capacity.

Q18. What happens if both peers use TURN?

Media goes from browser A to its TURN allocation and on to browser B's allocation or directly to B. It works; it's just the longest path, so latency is highest. ICE still picks the best working pair it finds.

FAILURE SCENARIOS

Q19. What happens if TURN is not configured?

Calls still work wherever a direct connection is possible, which is most cases. For users behind strict networks the call fails; our app tries ICE restarts and then shows a clear 'Unable to connect' message explaining that a TURN relay is needed, with a Try again button.

Q20. What happens if the TURN server goes down during a relayed call?

The media path breaks, so the connection becomes disconnected and then failed. Our app tries an ICE restart; if a direct path or another TURN server works, the call recovers, otherwise the user sees the failure screen. Configuring more than one TURN URL helps.

Q21. What if the TURN credentials are wrong?

The browser can't get relay candidates, so it behaves as if TURN weren't configured: direct connections still work, but restricted networks fail. Forcing relay mode in testing makes this obvious immediately.

Q22. How did you design the user experience for connection failures?

The status bar shows the current state in words, 'Reconnecting…' appears during short drops, and a final failure gives a plain explanation and a retry button instead of a frozen video. The appointment stays joinable, so they can try again.

Q23. What's the difference between the signaling connection failing and the media connection failing?

If the WebSocket drops, the media can keep flowing for a while because it doesn't use the server; our app reconnects the socket and renegotiates. If the media path fails, the WebSocket is still fine, and we use it to perform an ICE restart. They're independent, and the app handles each separately.

SCALE AND OPERATIONS

Q24. How would you deploy TURN for production?

Run TURN servers in the regions where users are, with public IPs, ports for UDP, TCP and TLS on 443, time-limited credentials and monitoring of bandwidth. Several servers listed together give redundancy.

Q25. Why does TURN cost more than STUN?

STUN answers a few tiny packets per call. TURN carries the entire video stream for every relayed call for its whole duration, so its cost scales with bandwidth used.

Q26. If you had many simultaneous consultations, what would happen to TURN?

Only the relayed share uses it, but that share grows with users. You'd monitor bandwidth and add TURN servers horizontally. The API servers aren't affected because media never passes through them.

Q27. Could you force all calls through TURN for privacy?

Yes, by setting the transport policy to relay. Then peers never see each other's IP addresses. The cost is higher latency and bandwidth for every call, so it's a trade-off a clinic might choose for privacy reasons.

Q28. Does TURN help with more than two participants?

TURN only relays between peers; it doesn't mix or route streams for a group. For multi-party consultations, like a doctor, patient and interpreter, you'd use a media server that receives each stream once and forwards it to the others.

Q29. How would you test that TURN works before relying on it?

Configure the TURN server, set ICE_TRANSPORT_POLICY to relay, join a call from two browsers, and confirm the status shows 'Relayed via TURN'. Then switch the policy back to all for normal use.

Q30. Summarize STUN, TURN and signaling together.

Signaling over our WebSocket exchanges the offer, the answer and the candidates. STUN helps each browser find its public address so they can try a direct path. If no direct path works, TURN relays the encrypted media. In the end the media flows over WebRTC, not through our backend, except through TURN when necessary.

Project-Specific Architecture Questions

Q1. Why did you build this application?

I wanted a project that solves a real workflow problem end to end, not just a demo. Telemedicine combines several hard parts: role-based access to sensitive data, scheduling with concurrency, real-time video, and now AI that has to be used responsibly. And the documentation bottleneck is a real, practical pain point for doctors.

Q2. Explain the complete system architecture.

A React single-page app talks to a FastAPI backend over REST for normal data, and over two WebSockets: one pushes notifications, the other does WebRTC signaling. MySQL stores everything. The video and audio go directly between the two browsers over WebRTC, using STUN to find a direct path and TURN as a relay fallback. For documentation, the backend calls Groq's Whisper API and Google's Gemini API in a background task.

Q3. What does feature-first structure mean, and why did you choose it?

Code is grouped by business feature rather than by technical type. Everything about appointments is in one appointments folder: model, schemas, service and router in the backend; pages, components and api.js in the frontend. It makes changes local and the code easier to navigate than one big models folder and one big routes folder.

Q4. What are the main database tables and how are they related?

users holds every login with a role. doctor_profiles and patient_profiles are one-to-one with users. Doctors have availability windows and days off. appointments link a patient user and a doctor user. Each appointment has at most one consultation, and each consultation at most one documentation record. Notifications belong to a user, and specializations are referenced by doctor profiles.

Q5. Explain the appointment lifecycle.

A booking starts as pending and already holds the slot. The doctor accepts it, making it confirmed, or declines it, making it rejected and freeing the slot. Either side can cancel a pending or confirmed appointment before it starts. After the video consultation ends, it becomes completed.

Q6. How are available time slots calculated?

They're not stored. For a date, I take the doctor's weekly windows, for example Monday 9:00 to 12:30, split them into slots of the doctor's consultation length, and remove slots on days off, slots overlapping pending or confirmed appointments, and slots that have already started. The same function is used both to show slots and to validate a booking, so they always agree.

Q7. Why compute slots instead of storing them in a table?

If slots were rows, every schedule change would mean creating and deleting rows and keeping them consistent with bookings. Computing them from the schedule means a change in hours takes effect immediately, and there's nothing to get out of sync.

Q8. How exactly do you prevent double booking?

Inside one transaction, the booking first locks the doctor's profile row with SELECT ... FOR UPDATE, then rechecks that the slot is still free, then inserts the appointment and commits. A second request for the same doctor waits on the lock, then sees the first appointment and returns 409. I also set the isolation level to READ COMMITTED so the recheck sees the latest committed data rather than an older snapshot.

Q9. How do you handle time zones in scheduling?

Doctors enter hours in the clinic's time zone, configured in the environment. The backend converts slots to UTC and stores everything in UTC. The browser displays times in the user's local time zone.

Q10. Explain the consultation lifecycle.

When the first participant opens the room within the join window, a consultation record is created with a unique room id and status waiting, and the other participant gets a notification. When both are in, it becomes active and the start time is recorded. When either ends it, it becomes ended, and the appointment becomes completed.

Q11. When can users join the video room?

From 15 minutes before the start until 15 minutes after the scheduled end, configurable. A consultation that's already active can always be rejoined. The server calculates a can_join flag, so the frontend never decides this on its own.

Q12. How do real-time notifications work end to end?

When something happens, like a booking, the backend saves a notification in the database and pushes it over that user's notification WebSocket if they're online. The frontend adds it to the bell, shows a toast and lets the current page reload its data. If the user was offline, they see it next time they load the list.

Q13. Why use WebSocket for notifications instead of polling?

Polling would mean every logged-in user asking the server every few seconds, mostly getting nothing new. With a WebSocket the server pushes only when something changes, so updates are instant and cheap. We already needed WebSockets for signaling, so it reuses the same infrastructure.

Q14. How does the frontend talk to the backend in development?

Vite's dev server proxies /api, /ws and /uploads to FastAPI on port 8000. The browser only talks to localhost:5173, so there are no cross-origin issues and WebSocket URLs are simply built from the page's own address.

Q15. What would you change if you started again?

I'd add database migrations from day one and plan the in-memory WebSocket state as shareable from the start, since those are the first things to change for production. Otherwise the feature-first structure worked well.

AI, Whisper and Gemini Questions

WHY AI

Q1. What business problem made you introduce speech-to-text?

Documentation takes part of the doctor's limited consultation time. To automate any of it, the system first needs the conversation as text. Speech-to-text turns the recorded consultation into a transcript, which is the input for the structured summary.

Q2. Why not just let the doctor type the notes?

They still can: manual notes always work. But typing during the call takes attention away from the patient, and typing afterwards takes extra time and relies on memory. A draft that's already 80 percent done turns writing into reviewing, which is much faster.

Q3. What is Whisper?

Whisper is a speech recognition model trained on a large amount of multilingual audio. It transcribes speech into text, detects the language and gives timestamps for each segment.

Q4. Why Whisper for this project?

It's strong on multilingual and code-mixed speech, which matters for Hindi and Hinglish consultations, and it handles background noise and accents reasonably well. It also returns segment timestamps, which I need to interleave the doctor's and the patient's speech.

Q5. Why use a cloud API instead of running Whisper locally?

Running Whisper well needs a strong GPU, and performance would depend on whatever machine the backend runs on. A cloud API gives consistent speed without that hardware, and the backend stays a lightweight web server.

Q6. Why Groq specifically?

Groq hosts Whisper with very fast inference, so a consultation recording is transcribed in seconds, and it has a free tier that is practical for development. It uses an OpenAI-compatible transcription endpoint, which keeps the integration simple.

Q7. Which Whisper model do you use and why?

whisper-large-v3, Groq's most accurate multilingual Whisper model, because accuracy on Hindi and Hinglish matters more than a small speed gain. The faster large-v3-turbo can be selected through an environment variable if cost or speed becomes more important.

Q8. What is Gemini used for?

Turning the transcript into a structured clinical summary: chief complaint, symptoms, relevant history, current medications, allergies, observations, the assessment the doctor discussed, the treatment plan and follow-up instructions.

Q9. Why Gemini?

It follows instructions well, supports a JSON response schema so the output always has exactly the fields I expect, understands Hinglish input, and has a free tier. I use the gemini-2.5-flash model, which is fast enough for this kind of extraction.

HOW IT WORKS

Q10. Explain the AI documentation pipeline step by step.

During the call the doctor presses Record and the patient sees an indicator. The doctor's browser records two tracks: the doctor's microphone and the patient's incoming audio. When the call ends they're uploaded; the backend saves them privately, returns immediately, and a background task sends each file to Groq. The segments are merged by time into Doctor and Patient turns, the audio is deleted, and the transcript goes to Gemini. The draft is saved, and the doctor is notified to review, edit and approve it.

Q11. How do you know which lines were said by the doctor and which by the patient?

Each recording contains only one speaker: one is the doctor's own microphone, the other is the audio coming from the patient over WebRTC. So the speaker label comes from the file, not from guessing. Each file also stores when it started, so I shift its timestamps and merge both into one timeline.

Q12. Why not use a speaker diarization model?

Diarization guesses who is speaking from a single mixed recording and can make mistakes. In a video call we already have the two voices on separate tracks, so we get perfect labels for free and keep the system simpler.

Q13. How do you support Hinglish?

We don't force a language in Whisper and never use its translate option, so it transcribes what was actually said, including mixed sentences. The prompt to Whisper only lists medical vocabulary like medicine names and units to improve spelling. Then Gemini reads the mixed-language transcript and writes the clinical summary in English, keeping medicine names and doses exactly as spoken.

Q14. Why keep the transcript in the original language instead of translating it?

Translation at the transcription step can lose or change meaning, and the transcript is the evidence the doctor checks the summary against. Keeping it as spoken means the doctor can verify what the patient actually said.

Q15. How do you make sure Gemini's output has the right structure?

The request includes a response schema with exactly the nine sections as string fields, and asks for JSON. The backend parses it and keeps only the known sections. If the response isn't valid, it's treated as a failure the doctor can retry.

Q16. Why does the backend run the AI in the background?

Transcription and summarization can take tens of seconds. Holding an HTTP request open that long is fragile and would tie up the server. The upload returns 202 straight away, the processing runs in a background task, and the doctor's page shows each step's progress.

SAFETY AND CORRECTNESS

Q17. How do you handle hallucination risk?

In three layers. The prompt tells Gemini to use only the transcript, leave sections empty if not discussed, and never add diagnoses, medicines or doses that weren't said, with a low temperature. The doctor sees the transcript next to the draft and must review it. And only the doctor-approved version becomes the record.

Q18. What happens if the AI writes something that wasn't in the conversation?

The doctor removes or corrects it before approving. The original AI output is stored separately from the doctor's approved version, so we can always see what was changed. The patient never sees the AI draft.

Q19. Can the AI prescribe medication?

No. The plan section only records what the doctor said during the consultation, and nothing becomes part of the record without the doctor's approval. The AI is a documentation assistant, not a decision-maker.

Q20. Why must the doctor review the AI output?

Because the doctor is clinically and legally responsible for the record. Speech recognition can mishear a medicine name or a number, and a language model can misread context. A human check catches those errors before they reach the patient.

Q21. How are the API keys protected?

GROQ_API_KEY and GEMINI_API_KEY are only in the backend .env, which isn't committed. The browser never calls Groq or Gemini directly; it uploads audio to our backend, and the backend calls the APIs. The frontend only asks whether the feature is enabled and gets a yes or no.

Q22. Why must AI keys never be in the frontend?

Everything sent to the browser can be read by the user. A key in frontend code would let anyone use our account, run up costs, and send arbitrary data under our name. Keeping calls on the backend also lets us control who can trigger processing.

Q23. What privacy measures does the AI feature have?

Recording is off by default, the patient sees an indicator whenever it's on, audio is stored outside the public uploads folder and deleted right after transcription, transcripts and drafts are visible only to the consultation's doctor, and admins can't see them.

Q24. What happens if Groq or Gemini is unavailable?

The background task catches the error and marks the documentation as failed, recording whether it failed at transcription or summary, and the doctor gets a notification. They can press Retry, which continues from the failed stage, or write and approve the notes themselves. The consultation itself is unaffected.

Q25. What are the limitations of the AI feature?

Quality depends on audio quality and microphones. Rare medicine names can be misheard. The recording lives in the doctor's browser until the call ends. And it adds cost per consultation for API usage. All of that is acceptable because the doctor reviews the result.

Q26. How would you improve the AI feature later?

Show the draft live during the call, highlight low-confidence words for the doctor, let the doctor ask for a prescription draft in a fixed format, track how much doctors edit drafts to measure quality, and add a patient consent step before recording starts.

WebRTC Scenario Questions

Follow-up chain the interviewer is likely to use:

```
What is WebRTC?  ->  Why WebRTC here?  ->  How do the browsers connect?
   ->  What if they can't connect directly?  ->  Why TURN if you have STUN?
```

Q1. What is WebRTC?

WebRTC is a browser technology for real-time audio, video and data between browsers. It handles camera access, encoding, encryption and network traversal, and it's available in every modern browser without plugins.

Q2. Why did you use WebRTC?

We use WebRTC for the actual audio and video between the doctor and patient. It's designed for real-time peer-to-peer communication with low latency and built-in encryption, which suits a consultation. Because media goes directly between browsers, our server isn't loaded with video traffic.

Q3. Why do you need WebSocket if WebRTC carries the media?

Before two browsers can talk directly, they need to exchange connection details: what media they support and how they can be reached. WebRTC doesn't define how to exchange that, so we use our WebSocket as the signaling channel. Once connected, the media no longer uses it.

Q4. Why doesn't the WebSocket carry the audio and video?

WebSocket runs over TCP through our server. Video through it would add latency, fill our server's bandwidth for every call, and TCP's retransmissions cause stutter. WebRTC sends media over UDP directly between peers, which is built for live media.

Q5. What is SDP?

Session Description Protocol is a text description of a peer's media setup: audio and video, codecs, encryption fingerprints and network details. One side sends an offer SDP, the other replies with an answer SDP, and that agreement defines the call.

Q6. How does signaling work in your app?

Both users connect to /ws/consultations/{room_id} and authenticate. The first person waits; when the second joins, the first receives peer-joined and creates the offer. The second applies it and sends an answer. Both sides send ICE candidates as they find them, and the server just relays each message to the other participant.

Q7. Why does the person already in the room make the offer?

If both sides made offers at the same time, they'd collide, which is called glare. The server processes joins one at a time, so exactly one participant receives peer-joined and makes the offer. It's a simple rule that avoids that problem.

Q8. What happens if STUN fails?

Browsers fall back to their local addresses, which only work on the same network. Across networks the connection fails, our app tries ICE restarts and then shows 'Unable to connect' with a retry option and a note that a relay may be needed.

Q9. When does TURN become necessary?

When no direct path works: symmetric NAT, corporate firewalls, blocked UDP or some mobile networks. Then ICE uses the TURN relay candidate and the status bar in our app shows 'Relayed via TURN'.

Q10. What happens if the network changes during a call?

The connection state goes to disconnected. We wait four seconds, because short drops often recover, then the original offerer does an ICE restart, up to two times. If it can't recover, the user sees a failure screen with a retry button; the appointment stays joinable.

Q11. What happens if the other participant closes their tab?

Their WebSocket closes, the server sends peer-left to the remaining user, and their app closes the old peer connection and shows 'Waiting for ... to join'. When the person comes back, they're treated as a new arrival and the connection is set up again.

Q12. What happens if the signaling WebSocket disconnects mid-call?

Our app tries to reconnect it with increasing delays, up to five attempts. When it reconnects, the room logic runs again and the call is renegotiated. After five failed attempts, the user sees a message and can rejoin.

Q13. How do you handle mute and camera off?

We set track.enabled to false, which stops sending real audio or video without renegotiating the connection. We also send a small media-state message over the signaling socket, so the other side shows 'Camera is off' and a muted icon instead of a frozen black screen.

Q14. What happens when the user has no camera or denies permission?

If there's no camera, we automatically continue with audio only. If permission is denied, the lobby explains how to allow access and offers Try again or 'Join without camera and microphone'. The peer connection still receives the other person's audio and video.

Q15. How do you clean up when the user leaves?

The hook's cleanup stops all camera and microphone tracks, closes the peer connection, closes the WebSocket and clears timers. The server notices the closed socket and tells the other participant. A beforeunload warning also appears if the user tries to close the tab during a call.

Q16. Is the video call encrypted?

Yes. WebRTC media is always encrypted between the browsers. Even when relayed through TURN, the relay can't decrypt it. Signaling happens over our authenticated WebSocket, which in production would run over TLS.

Security Questions

Q1. What security concerns exist in a telemedicine application?

Medical data is among the most sensitive personal data. The main concerns are unauthorized access to records, impersonation of doctors, leaking data through APIs or logs, insecure video sessions, secrets in code, and AI services receiving data they shouldn't keep. Each part of the design addresses one of these.

Q2. How do you protect patient information?

Every endpoint requires authentication; queries are always filtered by the signed-in user; doctors only see patients they have an appointment with; admins don't see clinical data at all; responses use explicit schemas so extra fields can't leak; and patients only see doctor-approved documentation.

Q3. How do JWT and authorization work together?

The JWT proves who the user is. Authorization then checks the user's role from the database for the type of action, and ownership in the service for the specific record. A valid token is required but never sufficient on its own.

Q4. How do you prevent a patient from accessing another patient's history?

History endpoints don't accept a patient id at all; they use the id of the signed-in user. Requests for a specific appointment check that the user is its patient or doctor and return 404 otherwise. These rules are covered by automated tests.

Q5. How do you stop someone from joining a consultation they don't belong to?

The join endpoint checks the user is the appointment's patient or doctor, the appointment is confirmed and it's within the join window. The signaling socket checks again on connect that the user belongs to that room, and closes the connection with code 4403 otherwise. Room ids are random UUIDs.

Q6. How do you stop fake doctors?

New doctor accounts start as pending and are hidden from search and booking until an admin verifies their license and qualifications. Changing the license or specialization later sends them back to pending.

Q7. How are uploads secured?

Profile photos are type- and size-checked and saved with random names. Consultation audio is stored outside the public folder, never served over HTTP, size-limited, and deleted after transcription.

Q8. What would you add for production security?

HTTPS everywhere, rate limiting on login, refresh tokens or httpOnly cookies, audit logs of who viewed which record, encryption at rest for clinical data, time-limited TURN credentials and an explicit patient consent step for recording.

Failure and Troubleshooting Questions

Q1. What happens if the database is unavailable?

API requests that need data fail with a server error, and the health endpoint reports the database as unavailable. The frontend shows its error state with a retry button instead of crashing. A video call that is already connected keeps going, because media doesn't use the database.

Q2. What happens if the backend goes down?

Pages can't load data, notifications stop and the signaling socket disconnects. An already connected call can continue for a while because media is peer-to-peer; the frontend keeps trying to reconnect the sockets. In-progress AI processing is lost and can be retried.

Q3. What happens if a WebSocket disconnects?

For notifications, the app reconnects with exponential backoff and reloads the list after reconnecting, so nothing is missed because notifications are stored in the database. For signaling, it reconnects and renegotiates the call.

Q4. What happens if TURN is unavailable?

Direct connections still work for most users. Users on restrictive networks can't connect; they see a clear failure message and can retry.

Q5. What happens if Groq is unavailable?

Documentation fails at the transcription stage. The audio files are kept so a retry can resend them, the doctor is notified, and manual notes remain available.

Q6. What happens if Gemini is unavailable?

Documentation fails at the summary stage, but the transcript is already saved. The doctor can read the transcript, write the summary manually, or retry, which only repeats the summary step.

Q7. A patient says the Join button doesn't appear. How do you debug it?

Check the appointment is confirmed, not pending; check the current time is within 15 minutes of the start; check it wasn't already ended. The server's can_join flag follows these rules, and the appointment page rechecks it every 30 seconds when the start is near.

Q8. A doctor says patients can't find them in search. Why?

Most likely the doctor is still pending verification, rejected or suspended, or the account is deactivated. Only verified, active doctors are listed. The doctor's dashboard shows a banner explaining the status.

Q9. Two patients say they booked the same slot. Is that possible?

Not with the current design: bookings lock the doctor's row, recheck the slot and only then insert. One of them would have got a 'slot no longer available' message. I'd check whether they were actually different slots or different doctors.

Q10. The call connects but there's no audio. What would you check?

Whether the other side is muted, which our UI shows; whether the browser blocked autoplay, in which case the 'Enable audio' button appears; and whether the microphone permission was granted. The video element is kept mounted even when the camera is off, so audio isn't lost that way.

Scaling and Future Improvement Questions

Q1. How would you handle many simultaneous consultations?

The good news is that video doesn't go through our servers, so each call only costs a few signaling messages. The main limits are WebSocket connections per server, TURN bandwidth for relayed calls, and AI processing volume.

Q2. How would you scale the backend horizontally?

Run several FastAPI instances behind a load balancer. REST is stateless, so that part just works. The two in-memory parts, the notification hub and the room manager, would move to a shared store like Redis with publish/subscribe, so a message for a user reaches whichever server holds their socket.

Q3. Why is the in-memory room manager a scaling problem?

If the doctor's socket is on server A and the patient's on server B, server A can't relay the offer to B's socket because it only knows its own connections. A shared broker, or routing both users of a room to the same server, solves that.

Q4. How would you scale the AI processing?

Move it from in-process background tasks to a job queue with separate workers, automatic retries and rate limiting, so spikes after many consultations end at once don't affect the API.

Q5. How would you scale the database?

Indexes are already on the main lookup columns like doctor and start time. Next steps would be connection pooling tuning, read replicas for search and history pages, and caching doctor search results, since 'next available' is computed per request.

Q6. What features would you add next?

Payments, e-prescriptions with a structured format, patient consent capture for recording, reminders before appointments, follow-up booking from the visit summary, and multi-party calls for interpreters or family members.

Q7. What would you need for group consultations?

With more than two people, each browser sending video to every other one doesn't scale. A media server, an SFU, would receive each stream once and forward it to the others. Signaling would then go between each browser and the media server.

Final Rapid-Fire Revision

One line each. Read this the night before the interview.

Topic	One-line answer
Problem	Limited consultation time is partly spent on documentation; AI drafts notes so the doctor focuses on the patient.
Roles	Patient, doctor, admin; doctors are pending until an admin verifies their license.
Stack	React + Vite, FastAPI, MySQL, JWT, WebSocket signaling, WebRTC, STUN/TURN, Groq Whisper, Gemini.
Structure	Feature-first: each feature owns its routes, models, services, pages and API calls.
Slots	Computed from weekly hours minus days off, booked slots and the past; never stored.
Double booking	Row lock on the doctor (SELECT FOR UPDATE) + recheck + READ COMMITTED; 409 for the loser.
Appointment states	pending -> confirmed -> completed; or rejected / cancelled.
Consultation states	waiting -> active -> ended.
Join window	15 minutes before start to 15 minutes after end, computed by the server as can_join.
JWT	HS256, claims sub/role/iat/exp, 8 h expiry; user reloaded from DB each request.
Authorization	Role check (dependency) + ownership check (service); 404 for others' records.
Doctor data access	Only with a pending/confirmed/completed appointment with that patient.
WebSocket auth	First message {type: auth, token}; never in the URL; close 4401 if invalid.
WebRTC	Media between browsers; WebSocket only for signaling.
SDP	Offer/answer describing media and connection setup.
Who offers	The participant already waiting, when peer-joined arrives; prevents glare.
ICE	Collect candidates, exchange them, test pairs, pick a working path.
Early candidates	Queued until setRemoteDescription completes.
STUN	Finds the public address so peers can connect directly; carries no media.
TURN	Relays encrypted media when direct fails; fallback because of latency and cost.
Reconnect	disconnected -> wait 4 s -> ICE restart by the offerer (max 2) -> failure screen + retry.
Direct vs relay	Detected from getStats selected candidate pair and shown in the call status bar.
Notifications	Saved in DB + pushed over /ws/notifications; reconnect with backoff, then refetch.
Recording	Doctor's browser records doctor mic + patient audio separately; patient sees an indicator.
Speaker labels	From the separate recordings, merged by timestamp; no diarization model.
Whisper	Groq whisper-large-v3, language auto-detected, no translation, so Hinglish stays Hinglish.
Gemini	gemini-2.5-flash, JSON schema with 9 sections, low temperature, use only the transcript.
Human in the loop	AI output, draft and approved version stored separately; patient sees approved only.
AI failure	Status records the failed stage; retry resumes there; manual notes always available.

Topic	One-line answer
AI keys	Backend .env only; browser just asks whether the feature is enabled.
Scaling	Stateless REST scales out; move notification hub and room state to Redis; queue for AI jobs.

> Closing line if asked 'anything else?': *The part I'm most comfortable defending is the trade-offs: computing slots instead of storing them, locking to prevent double booking, keeping media off the server, and making the AI a reviewed draft rather than a decision-maker.*